1.Backend API


```
[{"_id":"6aacb6c49a7f123d0fc45ce8","name":"milk","category":"beerage","unitPrice":500,"minStockLevel":1
50,"maxStockLevel":200,"currentStock":0,"createdAt":"2026-09-18T03:57:56.679Z","updatedAt":"2026-10-
05T05:21:05.191Z","__v":0},
{"_id":"6ac334648d315174c1d77e82","name":"Spoon","category":"Kitchen","unitPrice":75,"minStockLevel":20
,"maxStockLevel":500,"currentStock":51,"createdAt":"2026-10-05T05:23:48.064Z","updatedAt":"2026-10-
05T05:25:43.926Z","__v":0},
{"_id":"6ac51e50b9946b5e7c32349c","name":"Keyboard","category":"Computer","unitPrice":500,"minStockLeve
l":10,"maxStockLevel":100,"currentStock":30,"createdAt":"2026-10-06T16:14:08.996Z","updatedAt":"2026-
10-06T16:26:14.946Z","__v":0}]
```

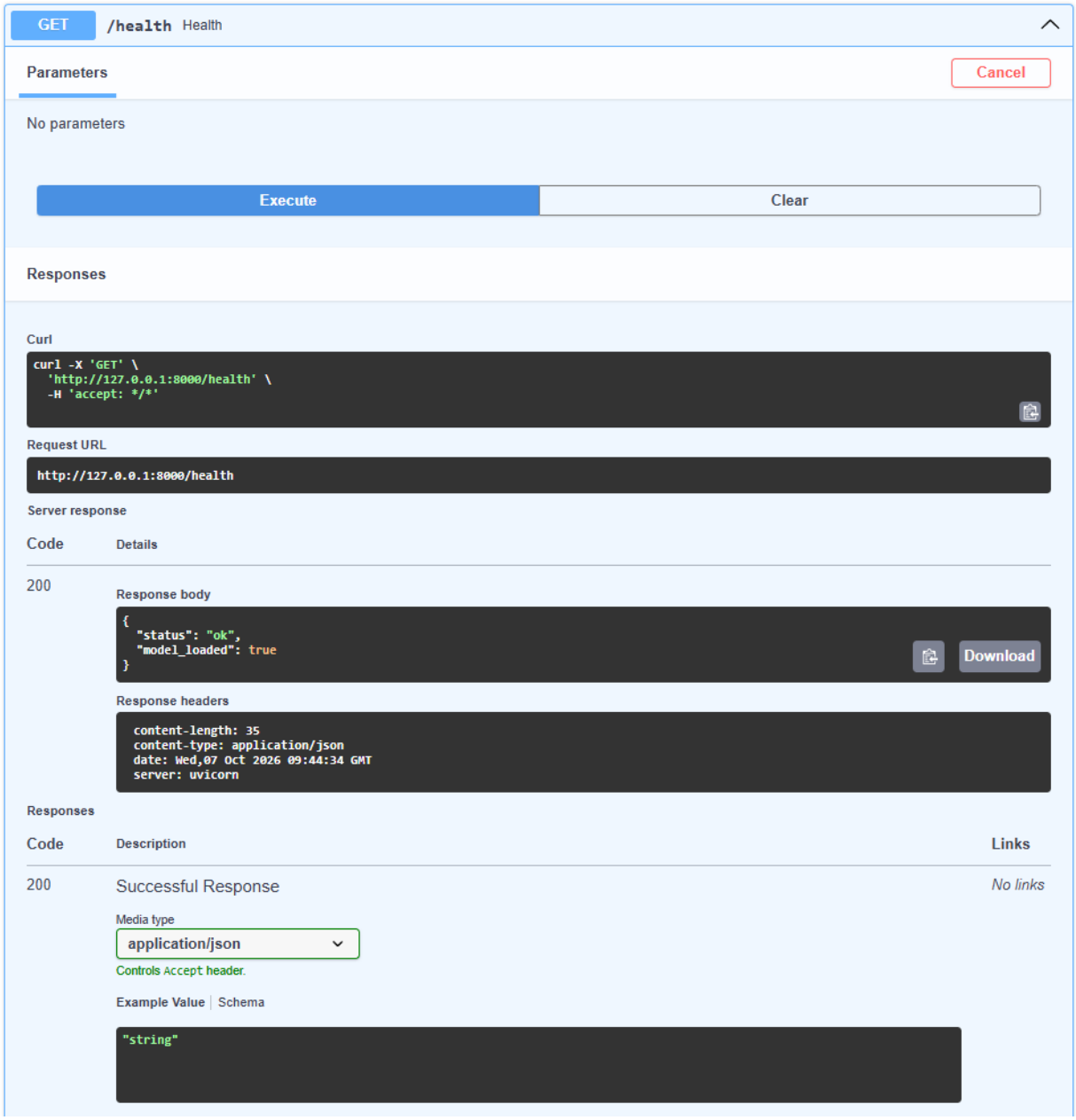

2.Backend Server and mongodb connection running

```
> erp-dss-backend@1.0.0 dev
[nodemon] watching extensions: js,mjs,cjs,json    [nodemon] watching extensions: js,mjs,cjs,[
nod[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
ERP DSS backend running on port 5000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
ERP DSS backend running on port 5000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
ERP DSS backend running on port 5000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
ERP DSS backend running on port 5000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
ing on port 5000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
ERP DSS backend running on port 5000
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
MongoDB connected: 127.0.0.1
```

3.FastAPI Services Running on port 8000

```
PS D:\University\Sem 4\AI\Group project\Project\erp-ai-dss\ai-service> python -m uvicorn app:
app --reload --host 127.0.0.1 --port 8000
INFO:     Will watch for changes in these directories: ['D:\\University\\Sem 4\\AI\\Group pro
ject\\Project\\erp-ai-dss\\ai-service']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [43076] using StatReload
C:\Users\ASUS\AppData\Roaming\Python\Python314\site-packages\sklearn\base.py:525: Inconsisten
tVersionWarning: Trying to unpickle estimator DecisionTreeRegressor from version 1.6.1 when u
sing version 1.9.1. This might lead to breaking code or invalid results. Use at your own risk
. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
C:\Users\ASUS\AppData\Roaming\Python\Python314\site-packages\sklearn\base.py:525: Inconsisten
tVersionWarning: Trying to unpickle estimator RandomForestRegressor from version 1.6.1 when u
sing version 1.9.1. This might lead to breaking code or invalid results. Use at your own risk
. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
INFO:     Started server process [45788]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

4. FastAPI prediction API Response


GET
/health Health
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/health' \
-H 'accept: */*'
Request URL
http://127.0.0.1:8000/health
Server response
Code
Details
200
Response body
{
"status": "ok",
"model_loaded": true
}
Download
Response headers
content-length: 35
content-type: application/json
date: Wed,07 Oct 2026 09:44:34 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"

default

GET / Home

Parameters | Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/
```

Server response

Code	Details
200	Response body

```
{
  "message": "ERP AI Service is running",
  "analyze": "/analyze",
  "predict": "/predict"
}
```

Download

Response headers

```
content-length: 81
content-type: application/json
date: Wed,07 Oct 2026 09:46:23 GMT
server: uvicorn
```

Responses

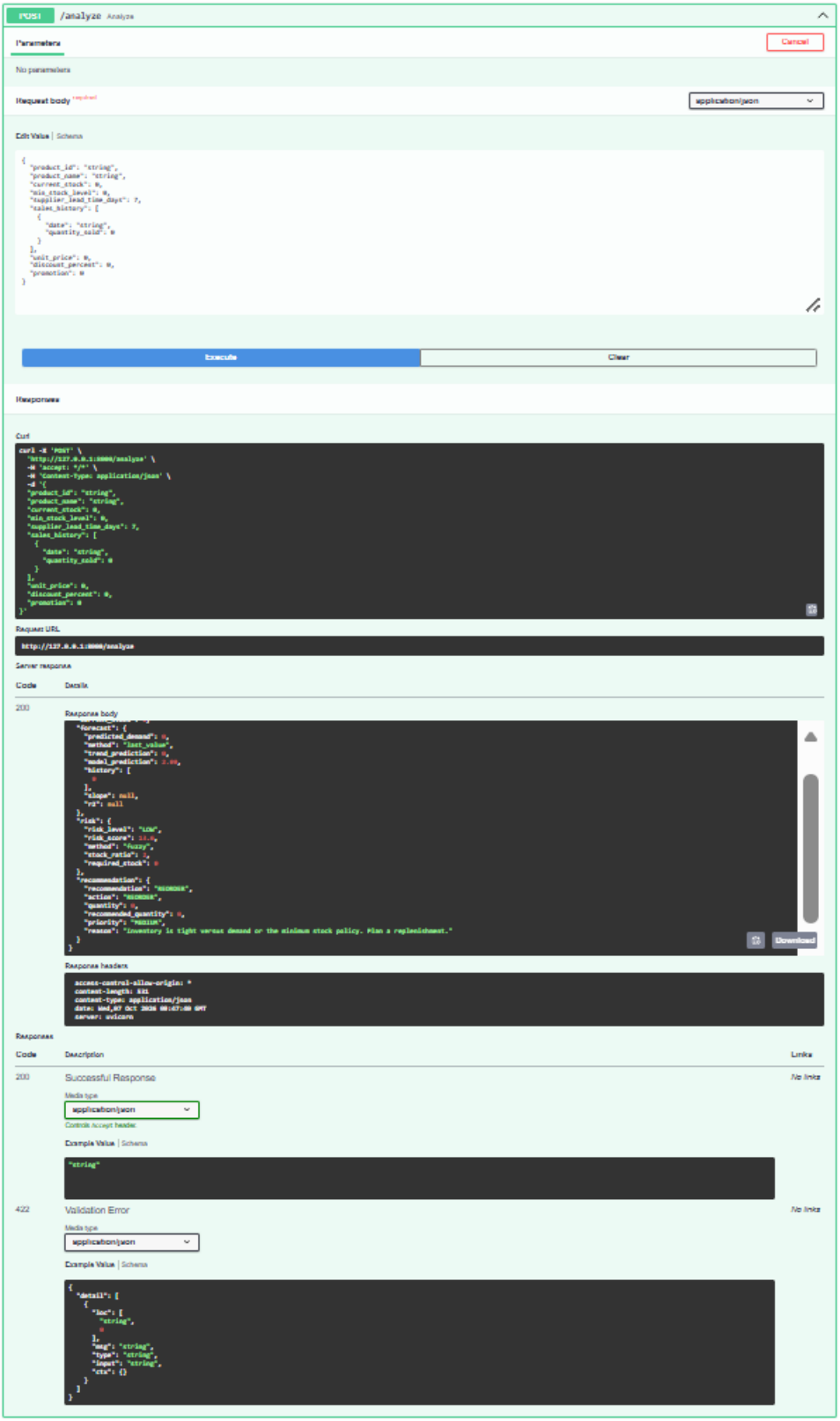
Code	Description	Links
200	Successful Response	*No links*

Media type: application/json

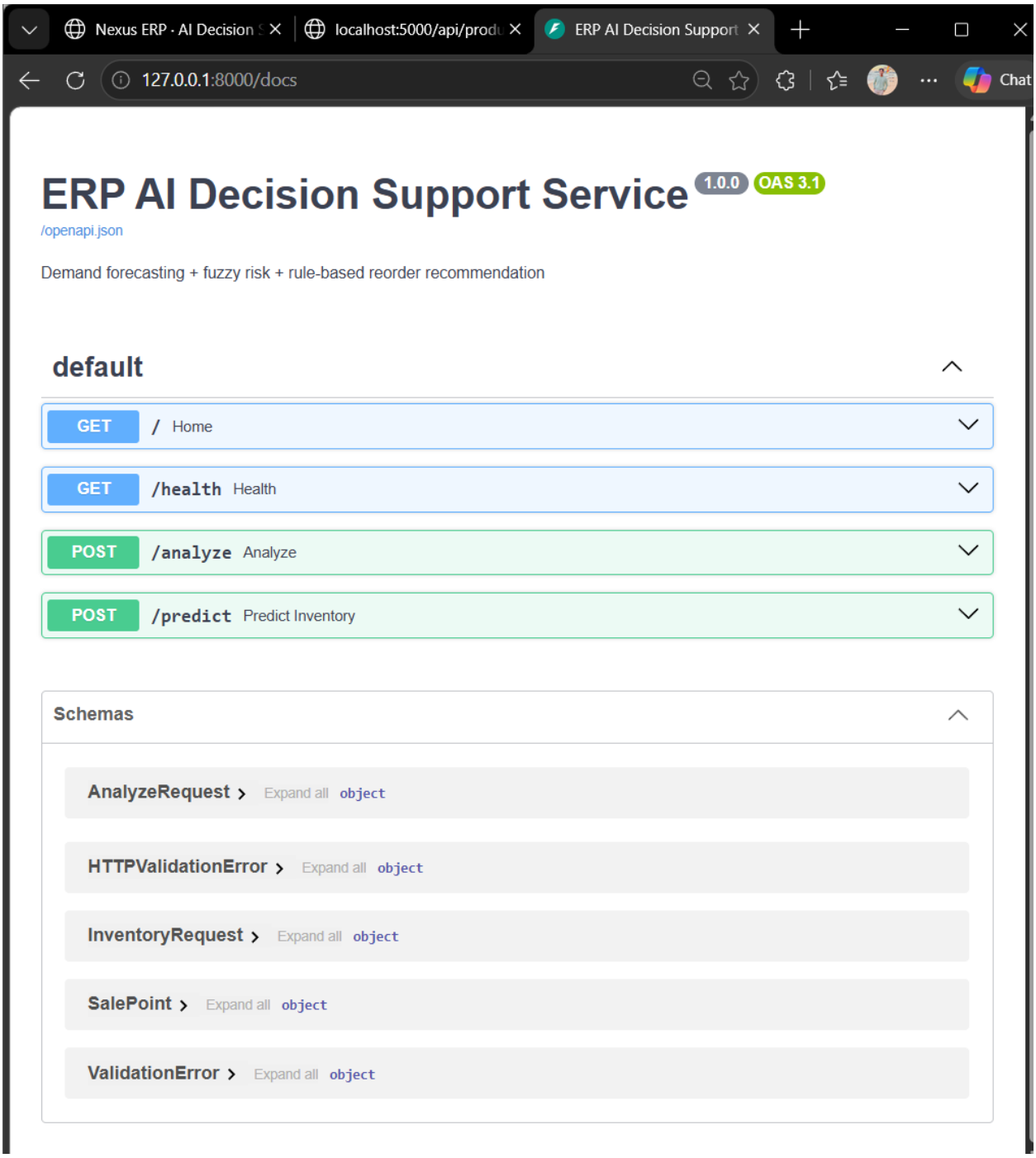
Controls `Accept` header.

Example Value | Schema

```
"string"
```


POST /analyze Analyze
Parameters
Cancel
No parameters
Request body required
application/json
Edit Value | Schema
{
"product_id": "string",
"product_name": "string",
"current_stock": 0,
"min_stock_level": 0,
"supplier_lead_time_days": 7,
"sales_history": [
{
"date": "string",
"quantity_sold": 0
}
],
"unit_price": 0,
"discount_percent": 0,
"promotion": 0
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/analyze' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"product_id": "string",
"product_name": "string",
"current_stock": 0,
"min_stock_level": 0,
"supplier_lead_time_days": 7,
"sales_history": [
{
"date": "string",
"quantity_sold": 0
}
],
"unit_price": 0,
"discount_percent": 0,
"promotion": 0
}'
Request URL
http://127.0.0.1:8000/analyze
Server response
Code
Details
200
Response body
"forecast": {
"predicted_demand": 0,
"method": "last_value",
"trend_prediction": 0,
"model_prediction": 2.99,
"history": [
0
],
"slope": null,
"r2": null
},
"risk": {
"risk_level": "LOW",
"risk_score": 33.5,
"method": "fuzzy",
"stock_ratio": 0,
"required_stock": 0
},
"recommendation": {
"recommendation": "REORDER",
"action": "REORDER",
"quantity": 0,
"recommended_quantity": 0,
"priority": "MEDIUM",
"reason": "Inventory is tight versus demand or the minimum stock policy. Plan a replenishment."
}
}
Download
Response headers
access-control-allow-origin: *
content-length: 621
content-type: application/json
date: Wed,07 Oct 2026 08:07:00 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"
422
Validation Error
No links
Media type
application/json
Example Value | Schema
{
"detail": [
{
"loc": [
"string",
0
],
"msg": "string",
"type": "string",
"input": "string",
"ctx": {}
}
]
}

5. FastAPI swagger API documentation



6.Fuzzy Risk assessment output

```
        return _ratio_risk(inventory_level, predicted_demand, supplier_python -c "from fuzzy_risk import assess_risk; pr
int(assess_risk(inventory_level=50, predicted_demand=600, supplier_lead_time=30))"
{'risk_level': 'HIGH', 'risk_score': 74.2, 'method': 'fuzzy', 'stock_ratio': 0.083, 'required_stock': 600.0}
PS D:\University\Sem 4\AI\Group project\Project\erp-ai-dss\ai-service>
```

7.Fuzzy Rules

```
rules = [
    ctrl.Rule(coverage_in["low"] & lead_in["long"], risk_out["CRITICAL"]),
    ctrl.Rule(coverage_in["low"] & lead_in["medium"], risk_out["HIGH"]),
    ctrl.Rule(coverage_in["low"] & lead_in["short"], risk_out["HIGH"]),
    ctrl.Rule(coverage_in["medium"] & lead_in["long"], risk_out["HIGH"]),
    ctrl.Rule(coverage_in["medium"] & lead_in["medium"], risk_out["MEDIUM"]),
    ctrl.Rule(coverage_in["medium"] & lead_in["short"], risk_out["MEDIUM"]),
    ctrl.Rule(coverage_in["high"] & lead_in["long"], risk_out["MEDIUM"]),
    ctrl.Rule(coverage_in["high"] & lead_in["medium"], risk_out["LOW"]),
    ctrl.Rule(coverage_in["high"] & lead_in["short"], risk_out["LOW"]),
]
```

Fuzzy Variables

```
coverage_in = ctrl.Antecedent(np.arange(0, 2.01, 0.01), "coverage")
lead_in = ctrl.Antecedent(np.arange(0, 60.01, 0.1), "lead")
risk_out = ctrl.Consequent(np.arange(0, 101, 1), "risk")
```

Membership Functions

```
coverage_in["low"] = fuzz.trimf(coverage_in.universe, [0, 0, 0.7])
coverage_in["medium"] = fuzz.trimf(coverage_in.universe, [0.4, 1.0, 1.4])
coverage_in["high"] = fuzz.trimf(coverage_in.universe, [1.0, 2.0, 2.0])

lead_in["short"] = fuzz.trimf(lead_in.universe, [0, 0, 14])
lead_in["medium"] = fuzz.trimf(lead_in.universe, [7, 21, 35])
lead_in["long"] = fuzz.trimf(lead_in.universe, [28, 60, 60])

risk_out["LOW"] = fuzz.trimf(risk_out.universe, [0, 0, 35])
risk_out["MEDIUM"] = fuzz.trimf(risk_out.universe, [20, 45, 70])
risk_out["HIGH"] = fuzz.trimf(risk_out.universe, [55, 75, 90])
risk_out["CRITICAL"] = fuzz.trimf(risk_out.universe, [80, 100, 100])
```

Model Training

```
==================================================
                MODEL EVALUATION
==================================================

Decision Tree Regressor
MAE       : 0.8867
RMSE      : 1.4024
R2 Score  : 0.9833

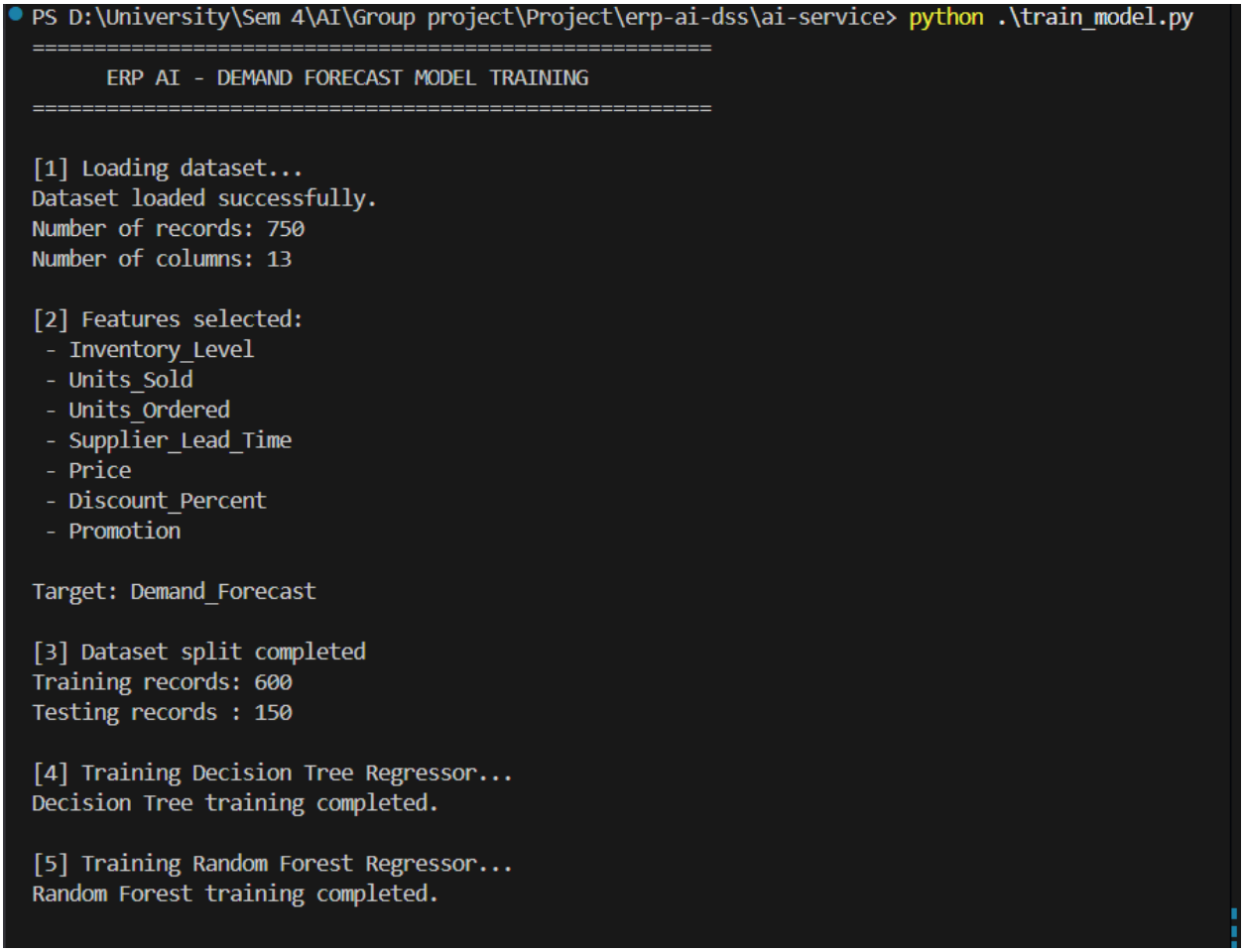
Random Forest Regressor
MAE       : 0.7687
RMSE      : 1.0074
R2 Score  : 0.9914

==================================================
SELECTED MODEL: Random Forest Regressor
==================================================

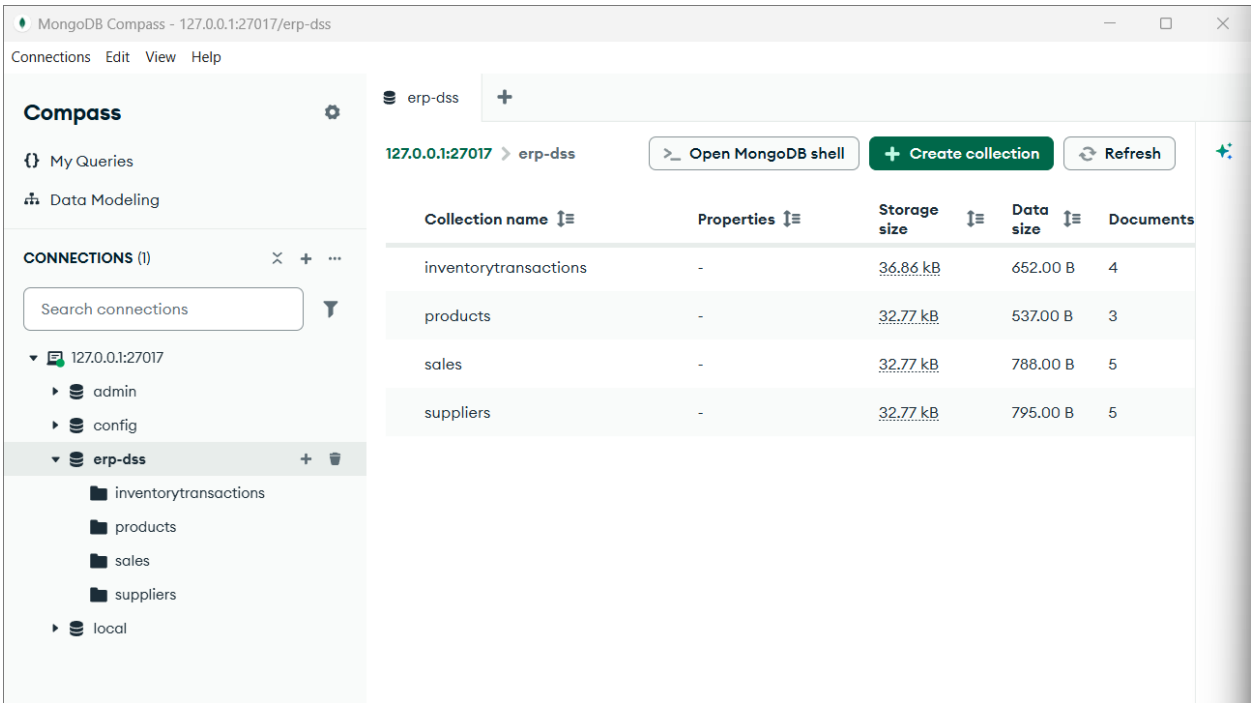
Model saved successfully:
models/trained_demand_model.pkl

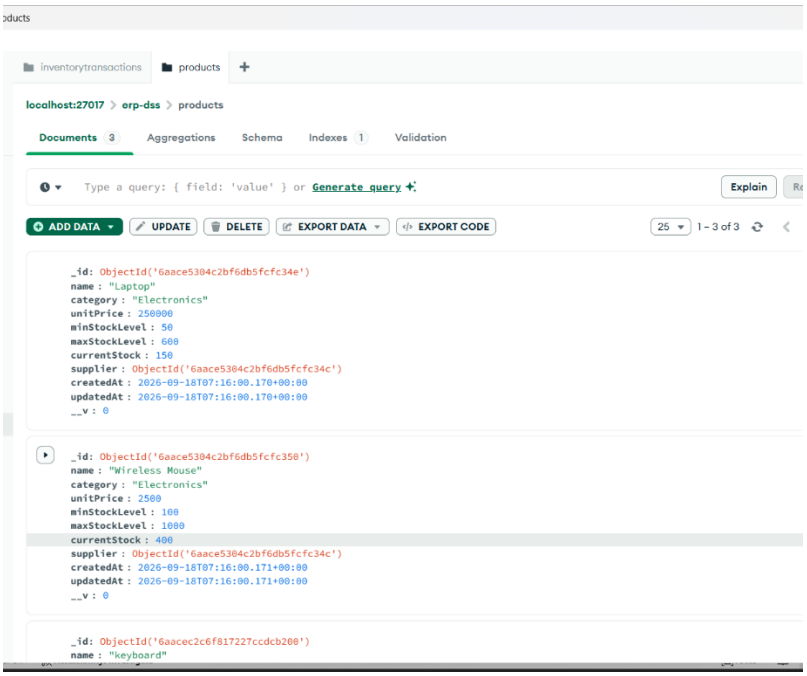
==================================================
                PREDICTION EXAMPLE
==================================================
Product             : Laptop
Inventory Level     : 42
Units Sold          : 8
Units Ordered       : 20
Supplier Lead Time  : 5 days
Price               : 250000
Discount            : 5%
Promotion           : Yes
Predicted Demand    : 10.28

Model training completed successfully.
PS D:\University\Sem 4\AI\Group project\Project\erp-ai-dss\ai-service>
```



MongoDB Database and collection





Reorder Calculation Logic

```
def recommend_reorder(
    inventory_level=None,
    predicted_demand=None,
    risk_level="LOW",
    min_stock_level=0,
    **kwargs,
):
    if inventory_level is None:
        inventory_level = kwargs.get("current_stock", 0)
    predicted_demand = predicted_demand if predicted_demand is not None else 0
    min_stock_level = kwargs.get("min_stock_level", min_stock_level) or 0
    lead_time = float(kwargs.get("supplier_lead_time_days", kwargs.get("supplier_lead_time",
7)) or 7)
    period_days = float(kwargs.get("period_days", 30) or 30)

    lead_demand = predicted_demand * (lead_time / period_days)
    target = max(lead_demand + min_stock_level, predicted_demand, min_stock_level)
    shortfall = max(target - inventory_level, 0)
    below_min = inventory_level <= min_stock_level

            "reason": reason,
        }

            "reason": reason,
        }
```

Reorder Quantity Calculation Logic

```
    if risk_level == "CRITICAL" or (below_min and predicted_demand > inventory_level):
        return pack(
            "URGENT_REORDER",
            max(shortfall, 1) if shortfall == 0 and below_min else shortfall,
            "HIGH",
            "Stock cannot cover forecasted demand across supplier lead time. Reorder before a stock-out.",
        )

    if risk_level == "HIGH":
        return pack(
            "REORDER",
            shortfall,
            "HIGH",
            "High stock-out risk versus lead-time demand. Place a replenishment order.",
        )

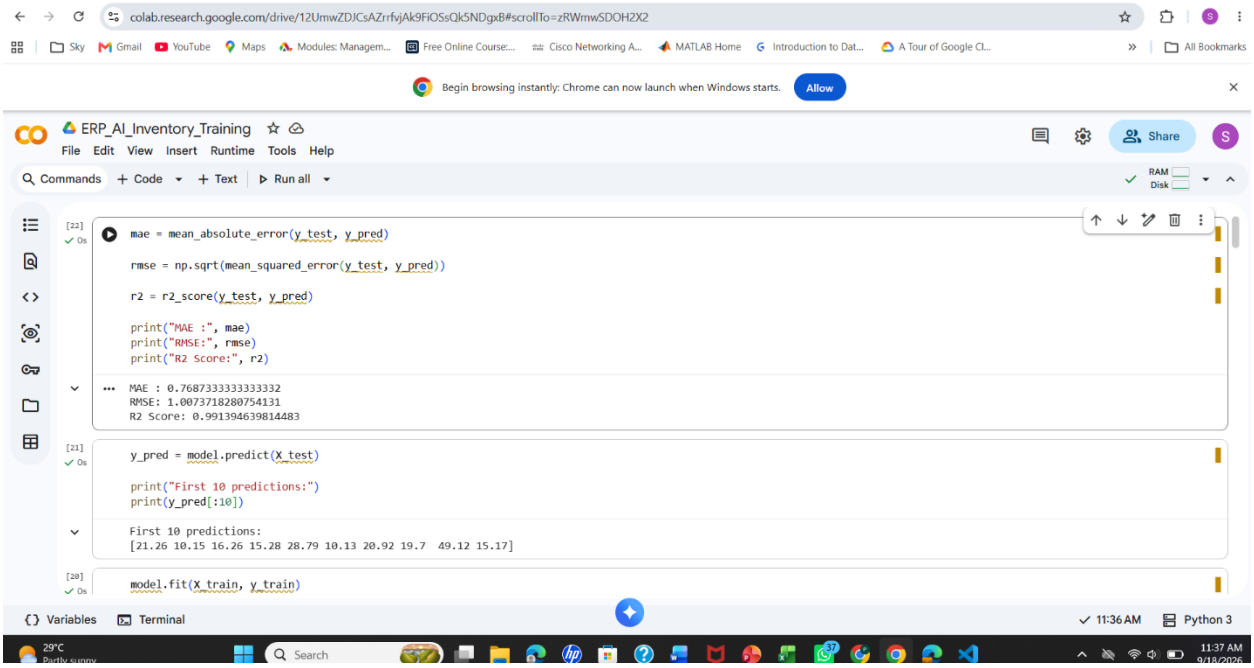
    if risk_level == "MEDIUM" or below_min:
        return pack(
            "REORDER",
            shortfall,
            "MEDIUM",
            "Inventory is tight versus demand or the minimum stock policy. Plan a replenishment.",
        )

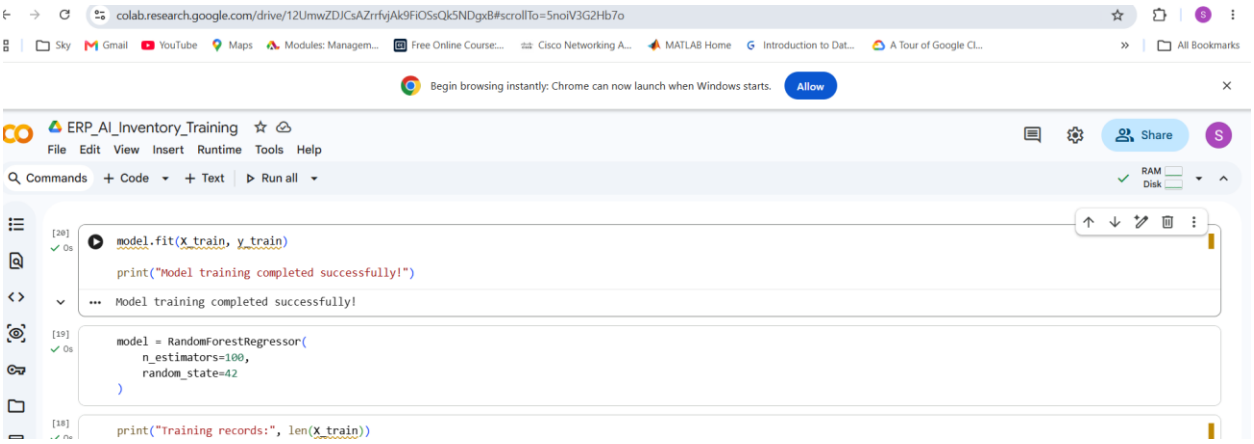
    return pack(
        "NO_REORDER",
        0,
        "LOW",
        "Current stock covers predicted lead-time demand and sits above the minimum stock level.",
    )
```

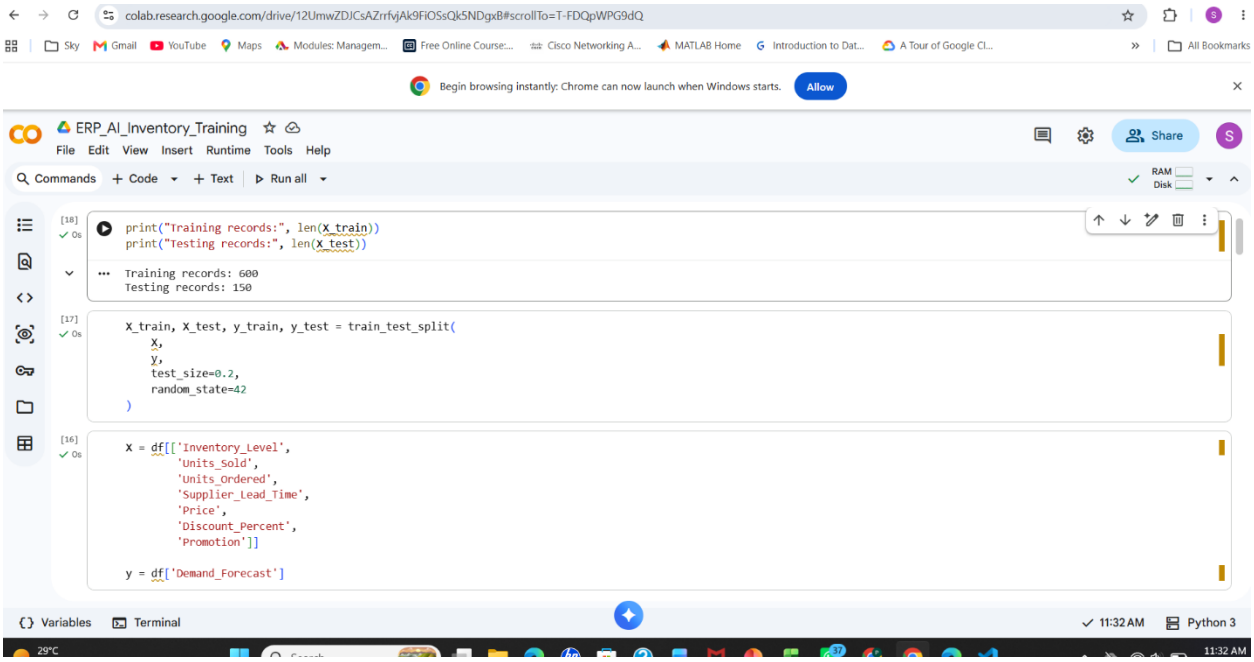
RuleBased Reorder decision

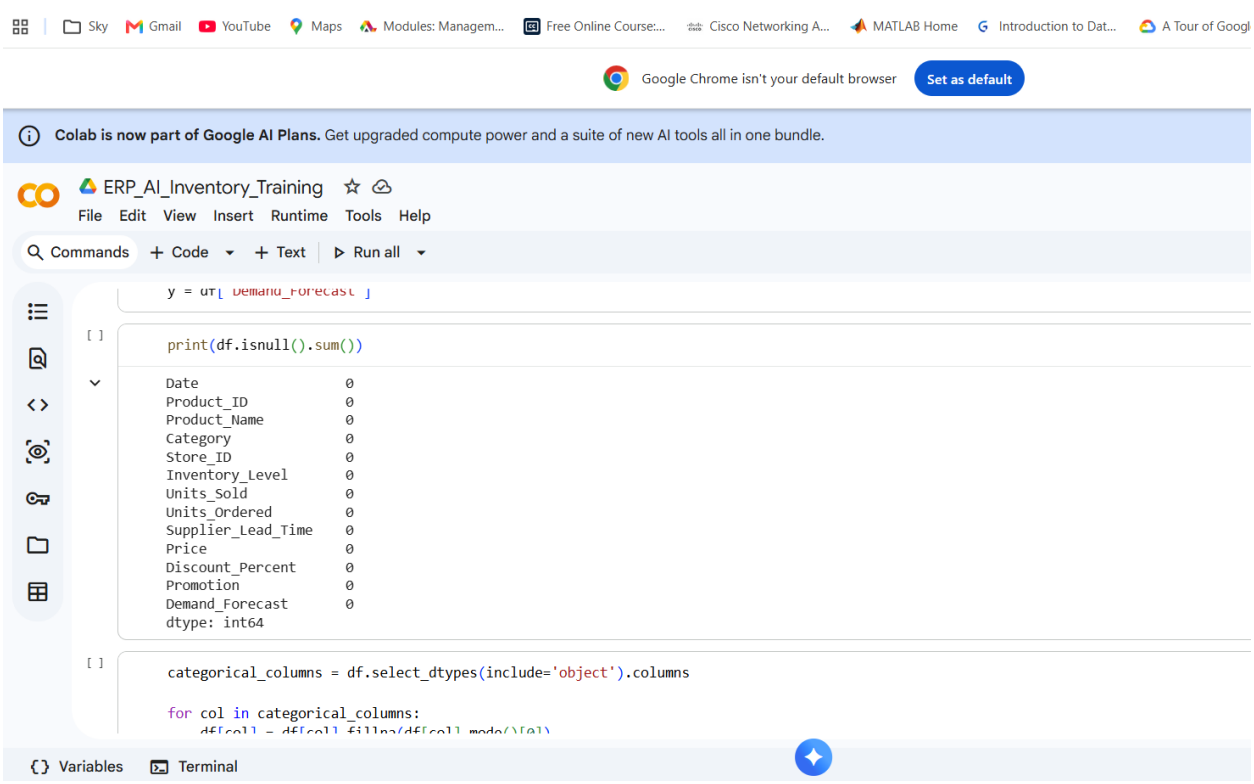
```
er; print(recommend_reorder(inventory_level=50, predicted_demand=600, risk_level='HIGH', min_stock_level=50, supplier_lead_time_days=30))"4\AI\Group project\Proje
ct\erp-ai-dss\ai-service>
{'recommendation': 'URGENT_REORDER', 'action': 'URGENT_REORDER', 'quantity': 600, 'recommended_quantity': 600, 'priority': 'HIGH', 'reason': 'Stock cannot cover f
orecasted demand across supplier lead time. Reorder before a stock-out.'}
PS D:\University\Sem 4\AI\Group project\Project\erp-ai-dss\ai-service>
```

Dataset Preparation


colab.research.google.com/drive/12UmwZDJCsAZrrfvjAk9FiOSsQk5NDgxB#scrollTo=zRWmwSDOH2X2
Begin browsing instantly: Chrome can now launch when Windows starts.
Allow
ERP_AI_Inventory_Training
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
[22] 0s
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print("MAE :", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
MAE : 0.7687333333333332
RMSE: 1.0073718280754131
R2 Score: 0.991394639814483
[21] 0s
y_pred = model.predict(X_test)
print("First 10 predictions:")
print(y_pred[:10])
First 10 predictions:
[21.26 10.15 16.26 15.28 28.79 10.13 20.92 19.7 49.12 15.17]
[20] 0s
model.fit(X_train, y_train)
Variables Terminal
11:36 AM Python 3
29°C Partly sunny
Search
11:37 AM 9/18/2026


colab.research.google.com/drive/12UmwZDJCsAZrrfvjAk9FiOSsQk5NDgxB#scrollTo=5noiV3G2Hb7o
Begin browsing instantly: Chrome can now launch when Windows starts.
Allow
ERP_AI_Inventory_Training
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
[20] 0s
model.fit(X_train, y_train)
print("Model training completed successfully!")
Model training completed successfully!
[19] 0s
model = RandomForestRegressor(
n_estimators=100,
random_state=42
)
[18]
print("Training records:", len(X_train))


colab.research.google.com/drive/12UmwZDJCsAZrrfvjAk9FiOSsQk5NDgxB#scrollTo=T-FDQpWPG9dQ
Begin browsing instantly: Chrome can now launch when Windows starts.
Allow
ERP_AI_Inventory_Training
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
[18] 0s
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
Training records: 600
Testing records: 150
[17] 0s
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42
)
[16] 0s
X = df[['Inventory_Level',
'Units_Sold',
'Units_Ordered',
'Supplier_Lead_Time',
'Price',
'Discount_Percent',
'Promotion']]
y = df['Demand_Forecast']
Variables
Terminal
11:32 AM
Python 3
29°C
11:32 AM


Sky
Gmail
YouTube
Maps
Modules: Managem...
Free Online Course:...
Cisco Networking A...
MATLAB Home
Introduction to Dat...
A Tour of Googl
Google Chrome isn't your default browser
Set as default
Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle.
ERP_AI_Inventory_Training
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
y = df[Demand_Forecast]
[ ]
print(df.isnull().sum())
Date 0
Product_ID 0
Product_Name 0
Category 0
Store_ID 0
Inventory_Level 0
Units_Sold 0
Units_Ordered 0
Supplier_Lead_Time 0
Price 0
Discount_Percent 0
Promotion 0
Demand_Forecast 0
dtype: int64
[ ]
categorical_columns = df.select_dtypes(include='object').columns
for col in categorical_columns:
Variables
Terminal

Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle.

ERP_AI_Inventory_Training

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
dtype: int64
```

[ ]
```
categorical_columns = df.select_dtypes(include='object').columns

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

[ ]
```
numeric_columns = df.select_dtypes(include=np.number).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())
```

[ ]
```
df = df.drop_duplicates()

print("Rows after removing duplicates:", len(df))
```

```
Rows after removing duplicates: 750
```

[ ]
```
df.describe()
```